

2: Supervised Learning

Date: January 20, 2026

1 What is Machine Learning?

The goal of machine learning is to design and analyze algorithms that improve performance on an underlying task given experience pertaining to the task. A standard machine learning pipeline looks as follows:



Figure 1: A typical machine learning pipeline.

The experience here refers to the available **training data**. Given the data, the **learning algorithm** takes the training data as input, and produces a **model** as output. We call the learning procedure that produces the model **training the model**. The **model** is a predictive (or decision-making) tool, and the performance of the model is evaluated based on some underlying **performance measure or criterion**. In order to specify a machine learning task, we need to specify all of the ingredients above:

1. What data are we using?
2. What learning algorithm should we run on this data?
3. How should we configure the learning algorithm, often by setting **hyperparameters**?
4. How do we evaluate the performance of the model output by the learning algorithm?

In many cases, the learning algorithm is some form of optimization procedure. Different algorithms lead to different performance measures, and generally stronger performance is preferred. There are however several other considerations and trade-offs, such as robustness, interpretability, computational and statistical efficiency of the model. We will touch upon some of these throughout the course. It is often useful to put your devil's advocate hat on, and question the limitations of different learning algorithms. When you use machine learning in the wild, even in the simplest settings that match perfectly the setup we are about to talk about, often the most difficult task will be deciding exactly what learning algorithm and configuration to use.

1.1 Machine Learning Paradigms

Machine learning algorithms can be generally categorized by kind of data they receive, and the type of knowledge they produce. While we will largely focus on supervised learning and unsupervised learning, it is helpful to be aware of other paradigms.

- *Supervised learning*: Training data consists of instance and label pairs. The goal is to predict well on future instances, for e.g. regression, classification.
- *Unsupervised learning*: Training data consists of only instances, no labels. The goal is to learn some patterns and structures in the data, for e.g. clustering, anomaly detection.
- *Semi-supervised learning*: Training data largely consists of unlabelled data, with access to a small amount of labelled data. The goal is to exploit both unlabelled and labelled data for better performance. This sits in between supervised and unsupervised learning.
- *Online learning*: Training data is received in an online fashion, that is, a single instance at a time. Here the learning algorithm is expected to make a prediction at each timestep prior to receiving the label.
- *Active learning*: This is a form of supervised learning where the learning algorithm gets to choose which instances it wants labels for with usually a cost associated with each labelling request. This is generally useful in settings where labelling cost is high. The goal is to predict well on future instances.
- *Reinforcement learning*: Unlike the above methods, no data is directly given to the learning algorithm (or agent). There is a set of states that an agent can be in, a set of actions that can be taken in each state which transition the agent to another state, and a ‘reward’ signal based on the state it transitions into. The goal is to learn a model (or policy), that determines which action the agent should take in each state, such that it maximizes the overall reward that is collected by following the policy from a fixed start state.

2 Supervised Learning

In a typical supervised learning problem, we are given data consisting of n labeled examples $\mathcal{D} = ((\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n))$. For each “labeled example,” $(\mathbf{x}_i, y_i)$, $\mathbf{x}_i$ denotes the **features** of that example, and y_i denotes the **label**. The features live in a common **feature space** or **input space** $\mathcal{X}$, and the labels live in a **label space** $\mathcal{Y}$. Typically the instance space is assumed to be a subset of $\mathbb{R}^d$, that is, the instances are d -dimensional vectors. Commonly, when given data on a computer, it will be in the form of a **feature matrix** $\mathbf{X} \in \mathbb{R}^{n \times d}$ and **label vector** $\mathbf{y} \in \mathcal{Y}^n$:

$$\mathbf{X} = \begin{bmatrix} - & \mathbf{x}_1 & - \\ - & \mathbf{x}_2 & - \\ & \vdots & \\ - & \mathbf{x}_n & - \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

Here are some examples making the above setup more concrete:

1. **Cat versus dog detector.** You are asked to train a model that takes an RGB image of size 1000x1000 pixels and predict whether the image contains a cat or a dog.
 - (a) **Dataset:** $\mathcal{D}$ consists of a series of images $\mathbf{x}_i$ and information about whether that image is of a cat or a dog y_i .
 - (b) **Features:** For this task $\mathbf{x}_i$ might be the pixels of the image themselves: a length $1000 \times 1000 \times 3$ vector (one entry per pixel for each of R, G, and B).
 - (c) **Labels:** For this task y_i might be 1 if the picture was of a cat, and -1 if the picture was of a dog.
 - (d) **Check:** Does it matter which class I label 1?
2. **Housing price predictor.** You are Zillow, and you are asked to train a model that takes all of the information a seller uploads about their home and predict the price of the house (the "Zestimate").
 - (a) **Dataset:** $\mathcal{D}$ consists of a series of descriptors $\mathbf{x}_i$ of houses that have sold previously and the price that house sold at y_i .
 - (b) **Features:** For this task $\mathbf{x}_i$ might consist of the number of bedrooms in the house, the square footage, the zipcode, etc.
 - (c) **Labels:** For this task y_i might be the final price the house sold at in dollars.

The goal is to learn from these examples a predictor $h : \mathcal{X} \rightarrow \mathcal{Y}$ that given an input $\mathbf{x}$ outputs a prediction $\hat{y} = h(\mathbf{x})$, with the goal that $\hat{y} \approx y$, where y denotes the true label of $\mathbf{x}$. Here are a few subtypes of supervised learning.

1. **Binary classification.** Our first example above was an example of what we call **binary classification**, where $\mathcal{Y} = \{-1, 1\}$. In binary classification, the goal is to predict the correct class for the input instance. Other examples of binary classification include spam filtering ($y = 1$ means "spam," $y = -1$ means "not spam"), disease detection ($y = 1$ means "disease present," $y = -1$ means "disease not present"), and so on.
2. **Regression.** In the second example above, the price of a house is much better modeled as being just a real number of dollars, $\mathcal{Y} \subseteq \mathbb{R}$. Even in settings like housing prediction where prices are technically usually only given to two decimal places (cents), it is much more convenient to work with the reals! The goal is again to predict the real-valued label associated with the instance, in this case the house price.
3. **Multi-class classification.** In multi-class classification, the label y can represent one of several discrete choices. For example, imagine if instead of merely wanting to distinguish cats versus dogs, we wanted our classifier above to tell us whether an image contained a dog, a cat, a kangaroo, or an otter. Here, we might let $\mathcal{Y} = \{0, 1, 2, 3\}$, where $y = 0$ denotes a dog, $y = 1$ denotes a cat, $y = 2$ a kangaroo and $y = 3$ an otter.

3 Training error

In these learning settings, a natural question is how well the model $h(\cdot)$ actually performs on data in the training set – that is, does $h(\mathbf{x}_i)$ actually equal y_i ? To evaluate this, we can measure the

training error. For binary classification, we might compute something like the fraction of points that $h(\cdot)$ classifies incorrectly.

$$\hat{R}_{0/1}(h) = \frac{1}{n} \sum_{i=1}^n \ell_{0/1}(h(\mathbf{x}_i), y_i),$$

where

$$\ell_{0/1}(\hat{y}, y) = \begin{cases} 0 & \text{if } \hat{y} = y \\ 1 & \text{otherwise.} \end{cases}$$

This probably seems like a lot of extraneous notation to measure something very simple, but we're going to reuse this a little later. Breaking things down a bit, $\ell_{0/1}(\cdot)$ just measures whether the prediction $\hat{y} = h(\mathbf{x}_i)$ equals the label y_i —in other words, whether $h(\cdot)$ correctly classified its input. Summing this for all training data points gives the total number of training data points classified incorrectly. $\hat{R}_{0/1}(h)$ divides this sum by n , and is therefore the fraction of points in the training data classified incorrectly. We refer to this as the **training error of a binary classifier** h .

For regression, things are a little more complicated. To see why, suppose the true price of a house with features $\mathbf{x}$ is $y = \$510000$. If our model predicts $h(\mathbf{x}) = \$509000$, then $\ell(h(\mathbf{x}), y) = 1$. If our model predicts $h(\mathbf{x}) = \$3.50$, then $\ell(h(\mathbf{x}), y) = 1$. The problem here is that, in a real world situation, we'd probably be pretty happy with a prediction of $\$509000$, and very unhappy with a prediction of $\$3.50$, but the calculation of error above does not capture this fact.

To circumvent this problem, we might instead measure our error using the **squared loss**:

$$\ell_{\text{sq}}(\hat{y}, y) = (\hat{y} - y)^2.$$

Under this definition of error, it's easy to see that $\$509000$ is indeed a better prediction for a label of $\$510000$ than $\$3.50$ is, as desired. This leads to the following notion of training error, the **mean squared training error**:

$$\hat{R}_{\text{sq}}(h) = \frac{1}{n} \sum_{i=1}^n \ell_{\text{sq}}(h(\mathbf{x}_i), y_i),$$

As we will study later in this course, a general strategy for learning algorithms is to minimize the training error. In other words, finding a model h from some set of models $\mathcal{H}$ that that minimizes the risk, a process known as **Empirical Risk Minimization**:

$$\min_{h \in \mathcal{H}} \hat{R}_{\text{sq}}(h)$$

4 Generalization

So far, we've defined what a training dataset might look like, what a learning task might be, and this nebulous concept of a “model” h that our learning algorithm might output. In order for *any* model to be useful, it will need to **generalize** beyond training data. To illustrate this point, suppose you are given a binary classification dataset $\mathcal{D}$, and further assume for now that each $\mathbf{x}_i$ is distinct (e.g., no two elements of your training dataset have exactly equal features). Consider the following trivial model:

$$h_{\text{bad}}(\mathbf{x}) = \begin{cases} y_i & \exists \mathbf{x}_i \in \mathcal{D} \text{ s.t. } \mathbf{x} = \mathbf{x}_i \\ 1 & \text{otherwise} \end{cases}$$

Obviously, this model achieves zero training error in all three learning settings we introduced! However, it's probably not a useful model. It just predicts 1 for any input not in the training data. The whole goal with machine learning is to achieve accurate predictions on data outside of the training data, a goal we call **generalization**. Unfortunately, there's a problem: unless we make some assumptions about how **test data**—data that we'd like the model $h(\cdot)$ to achieve high accuracy on but that it was not trained on—relates to the training data, this goal is pretty much impossible. Our test data could even be *adversarially* designed to fool our model.

In order to capture this, a commonly made, relatively mild assumption is the i.i.d. assumption. Here, we assume that the data (both training and test) is generated independently and identically from some underlying (unknown) distribution $\mathcal{P}$. Then we can define the risk of a predictor (generalization error) h on unseen examples, as the expected loss of a data point drawn from $\mathcal{P}$,

$$\underbrace{R(h)}_{\text{True Risk}} = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{P}} [\ell(h(\mathbf{x}), y)].$$

Note that the training error above is effectively just a monte carlo or finite sample approximation of the true risk! We often therefore refer to the training error as the **empirical risk**.

Observe the following tautology,

$$R(h) = \underbrace{(R(h) - \hat{R}(h))}_{\text{generalization gap}} + \hat{R}(h).$$

When we measured training error $\hat{R}(h)$, we only measured the second term. Therefore, generalization can be formalized as needing to ensure that the generalization gap is also small in order to get small generalization error. Note that this formalization makes it immediately mathematically clear why the trivial $h_{\text{bad}}(\mathbf{x})$ is a bad model. When we are forced to compute the expected loss over the data distribution and not just a finite sum, the true risk will be terrible.

Estimating the generalization gap. The fundamental problem with computing only training error is that it only measures the empirical risk or training error $\hat{R}(h)$, and gives us no way to estimate the generalization gap. To estimate the generalization gap, we divide our data $\mathcal{D}$ into a training and test set. We train on the training set and use the test set to evaluate the generalization gap. Commonly, 80% of the data might be used for training, for example. We will make this procedure much more detailed and rigorous later in the course. The key idea is that averaging the error over a test set will be an estimate of $R(h)$ without also being terms that are in $\hat{R}(h)$! This is because the test set is not used to train the model.

What is the best possible classifier? A natural question to ask is whether it is possible to find some classifier that achieves zero generalization error. Unfortunately, this is generally not possible: it's possible in binary classification for example that some instance $\mathbf{x}$ can sometimes appear with a +1 label and sometimes with a -1 label. Commonly, this happens if there is some inherent noise or uncertainty in the underlying true mapping between $\mathbf{x}$ and the label, or if crucial information is not captured by $\mathbf{x}$. In general, under the distribution $\mathcal{P}$, we will denote by $\eta(\mathbf{x})$ the conditional probability of the label +1 given $\mathbf{x}$:

$$\eta(\mathbf{x}) = \Pr(y = +1 \mid \mathbf{x})$$

Considering this conditional distribution lets us write the 0/1 generalization error in a more concrete form:

$$\begin{aligned}
R(h) &= \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{P}} [\ell_{0/1}(h(\mathbf{x}), y)] \\
&= \mathbb{E}_{\mathbf{x}} \left[\mathbb{E}_{y|\mathbf{x}} [\ell_{0/1}(h(\mathbf{x}), y)] \right] \\
&= \mathbb{E}_{\mathbf{x}} [\Pr(y = +1 \mid \mathbf{x}) \mathbf{1}(h(\mathbf{x}) = -1) + \Pr(y = -1 \mid \mathbf{x}) \mathbf{1}(h(\mathbf{x}) = 1)] \\
&= \mathbb{E}_{\mathbf{x}} [\eta(\mathbf{x}) \mathbf{1}(h(\mathbf{x}) = -1) + (1 - \eta(\mathbf{x})) \mathbf{1}(h(\mathbf{x}) = 1)]
\end{aligned}$$

The inner portion of this final term is basically saying: if $h(\mathbf{x}) = -1$, we incur no loss if the true label was -1 which happens with probability $(1 - \eta(\mathbf{x}))$ by definition. On the other hand, we incur a loss of 1 with probability $\eta(\mathbf{x})$.

What the optimal classifier does is then pretty simple in the basic binary classification setting: if $\eta(\mathbf{x}) > (1 - \eta(\mathbf{x}))$ (or equivalently, $\eta(\mathbf{x}) > 0.5$), we output 1. Otherwise, we output -1. Formally:

$$h^*(\mathbf{x}) = \begin{cases} +1 & \text{if } \eta(\mathbf{x}) > \frac{1}{2} \\ -1 & \text{otherwise} \end{cases}$$

We call this optimal classifier the **Bayes optimal classifier for $\mathcal{P}$** . It achieves the minimum possible error for $\mathcal{P}$, called the **Bayes error for $\mathcal{P}$** :

$$\begin{aligned}
R^* &= \min_{h: \mathcal{X} \rightarrow \pm 1} R(h) = \mathbb{E}_{\mathbf{x}} [\eta(\mathbf{x}) \mathbf{1}(h(\mathbf{x}) = -1) + (1 - \eta(\mathbf{x})) \mathbf{1}(h(\mathbf{x}) = 1)] \\
&= \min_{h: \mathcal{X} \rightarrow \pm 1} R(h) = \mathbb{E}_{\mathbf{x}} [\min(\eta(\mathbf{x}), 1 - \eta(\mathbf{x}))]
\end{aligned}$$

Here, the second equality comes from the fact that only one of $\mathbf{1}(h(\mathbf{x}) = -1)$ and $\mathbf{1}(h(\mathbf{x}) = 1)$ can possibly be 1. If the former is 1, it is multiplied by a loss of $\eta(\mathbf{x})$. If the latter is 1, it is multiplied by the loss of $1 - \eta(\mathbf{x})$. Obviously, the *optimal* classifier will simply pick the minimum of these two options for any input $\mathbf{x}$.